
PROJECT REQUIREMENT DOCUMENT

Project 3 — Agentic Match Analyst

Multi-Agent Orchestration Layer | Explainable AI Interface

1. Project Identity

Field	Detail
Sub-project title	Agentic Match Analyst — Hierarchical Multi-Agent Tactical Intelligence
Project type	Agentic AI system with tool-calling, grounded generation, and report synthesis
Role on resume	AI Engineer / LLM Specialist / Solutions Architect
Depends on	Project 1 outputs — semantic JSON, PostgreSQL, FastAPI endpoints
Priority	Third — build after Project 1 is stable and Project 2 is functional

2. What This Project Is

A multi-agent AI system embedded in the dashboard that acts as an intelligent football analyst. A user — coach, analyst, or researcher — can type a tactical question like:

"How did Team A's defensive compactness change after they conceded, and did their passing network density shift?"

The system does not hallucinate an answer. It routes the query through specialized agents, each of which runs deterministic calculations against the computed outputs from Project 1, then synthesizes a factually grounded, evidence-backed report.

This is not a chatbot wrapped around a static database. It is a Zero-Trust Memory multi-agent system where the core LLM never reasons on raw coordinates — it only sees structured semantic attributes computed by dedicated tool agents.

3. Why Multi-Agent and Not Single-Agent

A single agent handling both geometric calculations and language generation is susceptible to:

- Context window saturation on complex multi-step queries
- Spatial coordinate hallucination when reasoning about pitch positions
- Tool-calling confusion when one agent manages too many tool types

By partitioning responsibilities across five specialized agents, coordinate calculation is fully isolated from language generation. The LLM only ever sees clean structured outputs — never raw x/y values.

4. The Five Agents

Agent 1 — Orchestrator Agent

- Role: Primary interface. Parses semantic intent from user query, decomposes into subtasks, routes to correct specialist agents, manages session history
- Tools: Intent classifier, task router, session memory manager
- Input: Raw user tactical query
- Output: Structured task assignments for specialist agents

Agent 2 — Spatial Topology Agent

- Role: Executes all geometric calculations on tracking and positional data
- Tools: Centroid calculator, convex hull tool, Delaunay triangulation tool, Hungarian role-permutation tool
- Input: Match ID, phase range, player IDs from Orchestrator
- Output: Structured spatial attributes — compactness score, territorial coverage, role assignment confidence

Agent 3 — Network Analytics Agent

- Role: Operates on Time-Varying Graph representations, computes all graph metrics
- Tools: TVG centrality calculator, clustering coefficient tool, passing density tool, network evolution comparator
- Input: Match ID, phase labels from Orchestrator

- Output: Centrality scores, density values, community structure, temporal network diff

Agent 4 — Valuation Engine Agent

- Role: Interfaces with the deterministic PostgreSQL database for action valuation
- Tools: xT query tool, VAEP query tool, SQLite action lookup, phase-level aggregation tool
- Input: Player IDs, match ID, phase range
- Output: xT values, VAEP scores, possession outcome probabilities

Agent 5 — Report Synthesizer / LLM-as-a-Judge

- Role: Receives structured numeric outputs from all three specialist agents, drafts a coherent tactical report, validates factual alignment before presenting to user
- Tools: Report template engine, factuality checker, Likert validation scorer
- Input: Structured outputs from Agents 2, 3, and 4
- Output: Final explainable coach-style tactical report

5. Agent Workflow

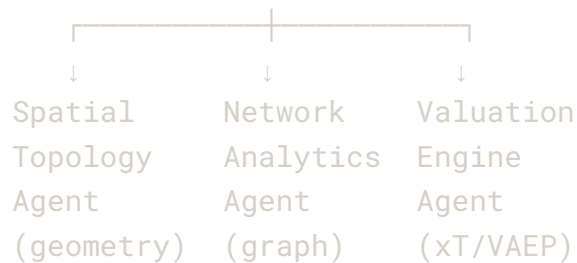
text

User Tactical Query



Orchestrator Agent

(parses intent, decomposes task)



Report Synthesizer

(LLM-as-a-Judge)

drafts + validates



Explainable Coach Report

shown in dashboard chat UI

Agents 2, 3, and 4 run in **parallel** using async execution — keeping response time under 5 seconds.

6. Zero-Trust Memory Paradigm

The core LLM — LLaMA 3.3 70B on Groq — never:

- Sees raw x/y coordinate values
- Performs arithmetic on positional data
- Guesses formation labels from player names

It only ever sees:

- Structured semantic attributes like `{"team_compactness": 0.73, "phase": "defensive_block", "centroid_shift_meters": 4.2}`
- Pre-computed graph metrics like `{"eigenvector_centrality_player_7": 0.91}`
- Validated action values like `{"xT_phase_total": 0.34, "VAEP_top_player": "player_9"}`

This is what eliminates hallucination and makes reports factually trustworthy.

7. System Architecture

Frontend Chat UI — React embedded in Project 2 dashboard

- Chat input box → user types tactical query
- Streaming response view → shows report as it generates
- Evidence panel → shows which tool outputs backed each claim
- Match context selector → which match and phase to analyze

LangGraph Agent Orchestration Layer — Cloud Run

- Orchestrator Agent node → intent parsing + task routing
- Spatial Topology Agent node → geometric tool execution
- Network Analytics Agent node → graph metric tools
- Valuation Engine Agent node → database query tools
- Report Synthesizer node → LLM drafting + validation
- LangGraph checkpoint → session state persistence

Tool Layer

- shapely tools → geometry calculations
- NetworkX tools → graph metrics
- SQLAlchemy tools → PostgreSQL xT/VAEP queries
- Custom semantic mapper → topology to structured attributes
- RAGAS evaluator → factuality and MRR scoring

LLM Layer

- Groq Cloud API → LLaMA 3.3 70B inference

Vector Store

- ChromaDB → semantic match summaries + report history
- sentence-transformers → local embedding generation

Storage

- PostgreSQL on Supabase → all match data, metrics, conversation history
- Redis on Upstash → cached agent tool results per match

8. Modes of Interaction

Mode 1 — Tactical Q&A (Chat)

User asks a question, agent returns a grounded answer with evidence.

Example: *"Which player had the highest passing centrality in the second half?"*

Mode 2 — Match Report Generation

User requests a full match report. System auto-generates a structured analyst-style document covering team shape, passing patterns, key players, phase analysis, and recommendations.

Mode 3 — Comparison Queries

User asks to compare two teams, two matches, or two players.

Example: *"Compare pressing intensity of Liverpool vs Arsenal in this match."*

Mode 4 — Phase Drill-Down

User selects a specific phase from the Project 2 timeline and asks the agent to explain what happened tactically during that phase.

9. Evaluation Plan

Component

Baseline

Metric

Retrieval quality	Single-agent RAG	MRR — Mean Reciprocal Rank
Factuality	Direct LLM zero-shot	Factuality check score via RAGAS
Report quality	Human analyst reports	Pairwise win-rate Turing-style
Hallucination rate	Single-agent with raw coordinates	Percentage coordinate errors in reports
Response latency	Single sequential agent	Time to first token + total response time

Human Expert Evaluation — Double-Blind Study

- 50 pairs of tactical summaries — one human-written, one agent-generated
 - Evaluated on 5-point Likert scale across four dimensions:
 1. Tactical validity — accuracy of identified tactical phases
 2. Actionable insights — practical value of coaching suggestions
 3. Structural coherence — logical flow and professional tone
 4. Grounded factuality — absence of statistical or conceptual hallucinations
 - Pairwise win-rate calculated to measure agent vs human preference
-

10. Tech Stack

Layer	Choice
Agent framework	LangGraph
LLM provider	Groq Cloud API
LLM model	LLaMA 3.3 70B
Vector database	ChromaDB
Embeddings	sentence-transformers
Tool libraries	shapely, NetworkX, SQLAlchemy
Evaluation	RAGAS
Session memory	LangGraph checkpointer + PostgreSQL
Cache	Redis on Upstash

Deployment	Google Cloud Run — separate container from Project 1
CI/CD	GitHub Actions
Chat UI	React component embedded in Project 2

11. Diagrams to Create

1. Multi-agent workflow diagram — all 5 agents, tools, and data flow
 2. Zero-Trust Memory diagram — what the LLM sees vs what tools compute
 3. Sequence diagram — one full user query end-to-end through all agents
 4. Component diagram — agent nodes, tool nodes, LLM, vector store, DB
 5. Deployment diagram — Cloud Run container for agents separate from backend
-

12. Deliverables

- Fully functional 5-agent LangGraph system deployed on Cloud Run
- Tactical Q&A mode with grounded evidence references
- Match report generation with structured analyst-style output
- Player and team comparison query support
- Phase drill-down integration with Project 2 timeline
- ChromaDB populated with semantic match summaries
- RAGAS evaluation pipeline with factuality scores logged
- Human evaluation protocol document — 50-pair Turing study
- Conversation history persisted per match session in PostgreSQL
- React chat UI embedded in Project 2 dashboard
- GitHub repository with clean agent module structure and README